

TOPIC-ONE

DevOps Fundamentals

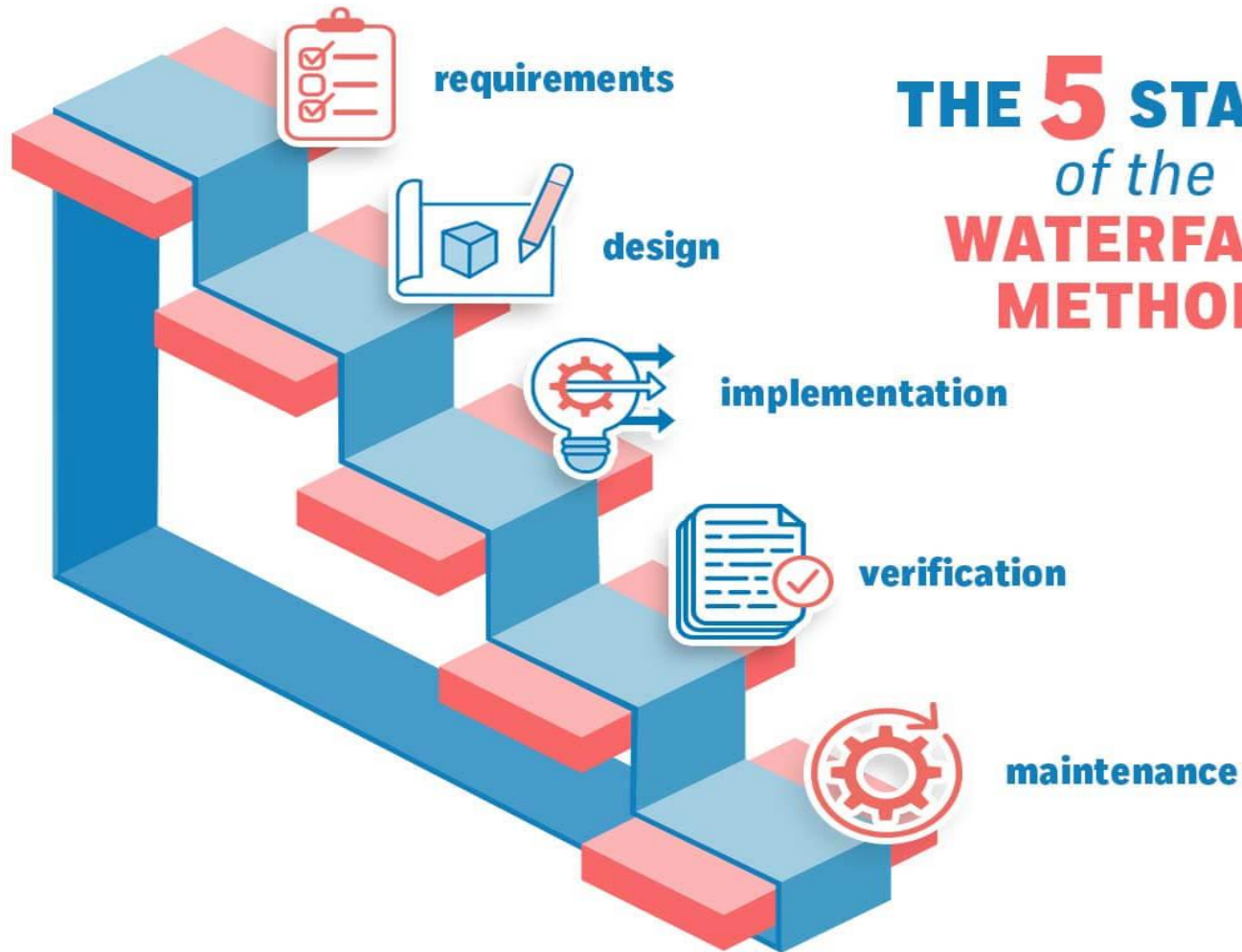
Author: [Arash Foroughi](#)





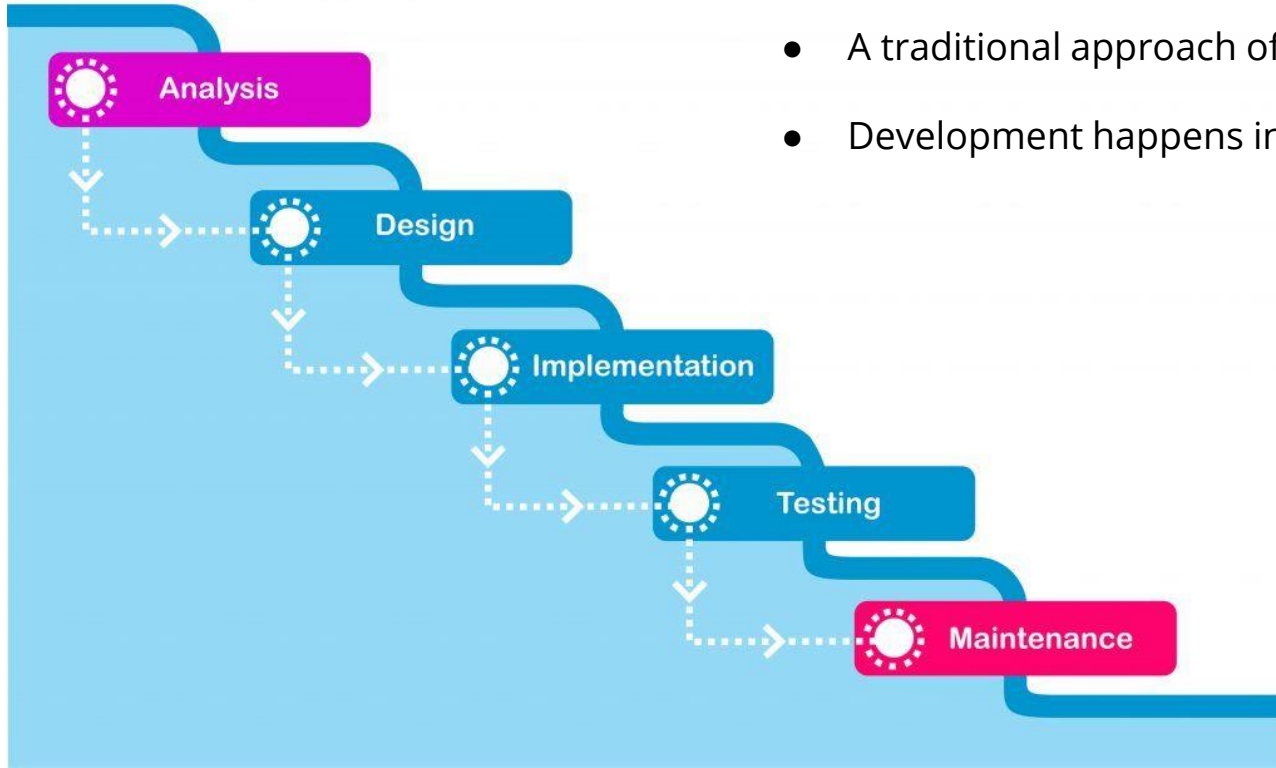
Waterfall Model





THE **5** STAGES *of the* **WATERFALL METHOD**

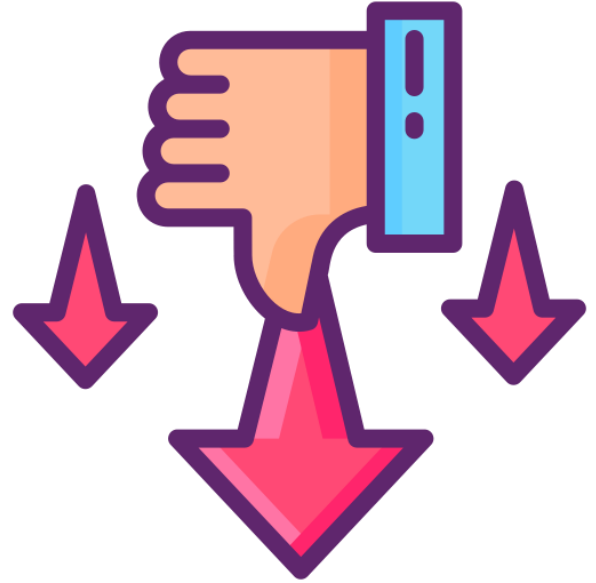
Waterfall Model



- A traditional approach of software development.
- Development happens in a step by step manner.

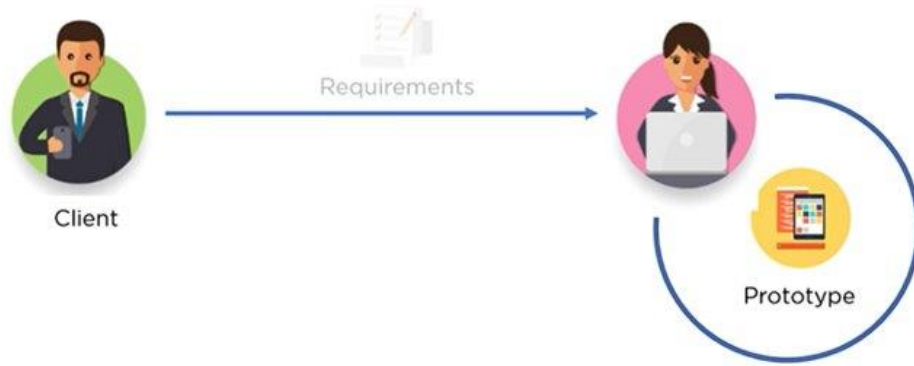
Waterfall Model Disadvantages

- Takes a very long time for developing code.
- It's difficult to deploy & give feedback of testing.
- Any new requirements will restart the cycle!
- If client is unhappy with product, again cycle will restart again!
- Until all requirements are not clear, project cannot start!

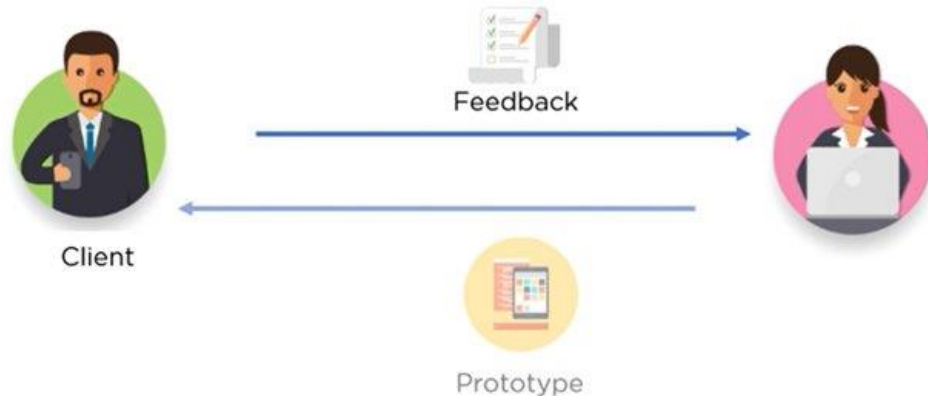


Agile Model

Agile Model



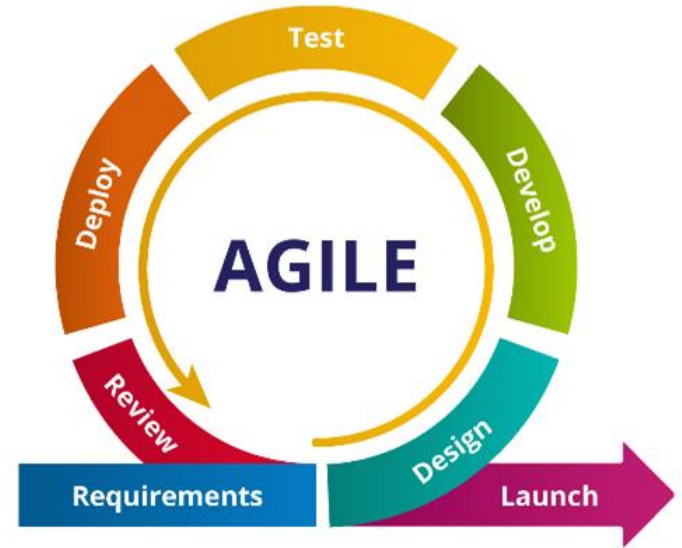
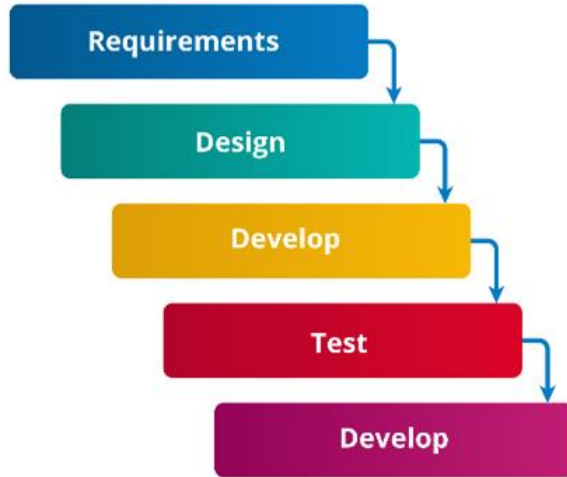
- Programmers create prototypes to clear customer requirements.



- Customer provides feedback & list of changes to be made

Waterfall vs. Agile

WATERFALL



Agile Model

- Process of development is broken down into small blocks called **sprints** or **Iteration**.



Agile Model Advantages & Disadvantages

ADVANTAGES

- Customer requirements are more clear because of the constant feedback.
- Product is delivered much faster as compared to waterfall model.

DISADVANTAGES

- Product gets tested only on developer computers & not on production systems.
- Developers & operations team-work will be broken.



What is **DevOps** integrated with **Agile**?



Agile Integration with DevOps



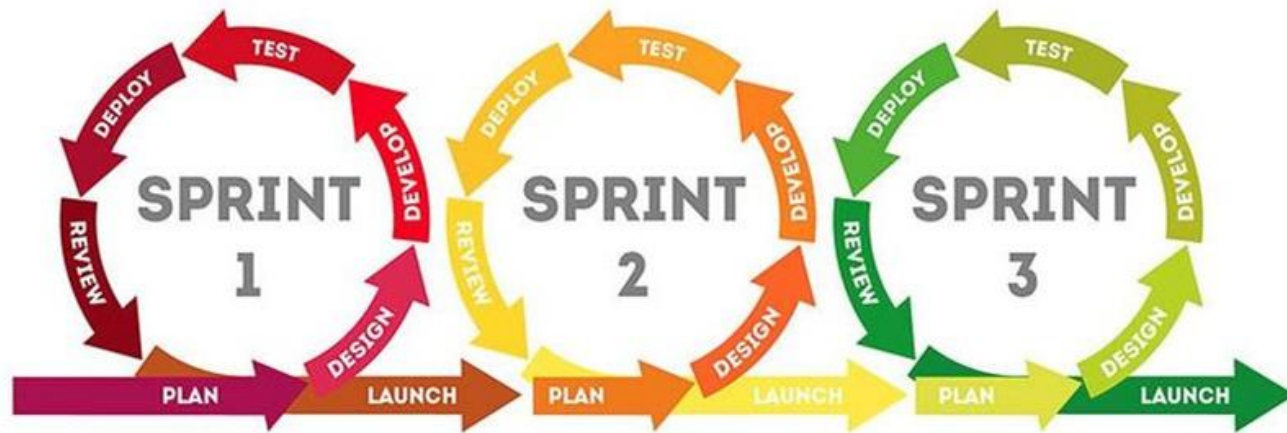
Agile addressed the gap between
Customers & Developers.

Agile Integration with DevOps



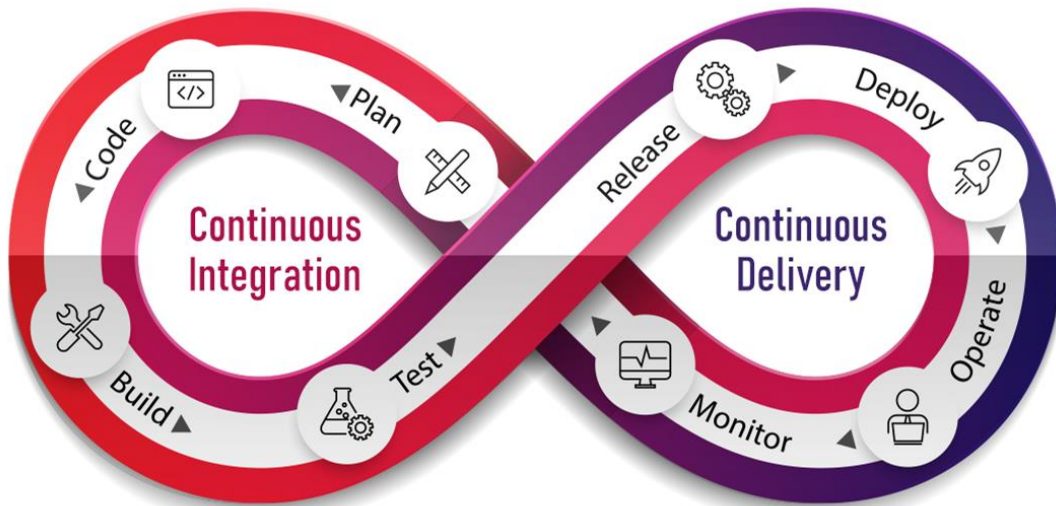
Agile addressed the gap between
Customers & Developers.

DevOps addressed the gap between
Developers & Operations.



Agile

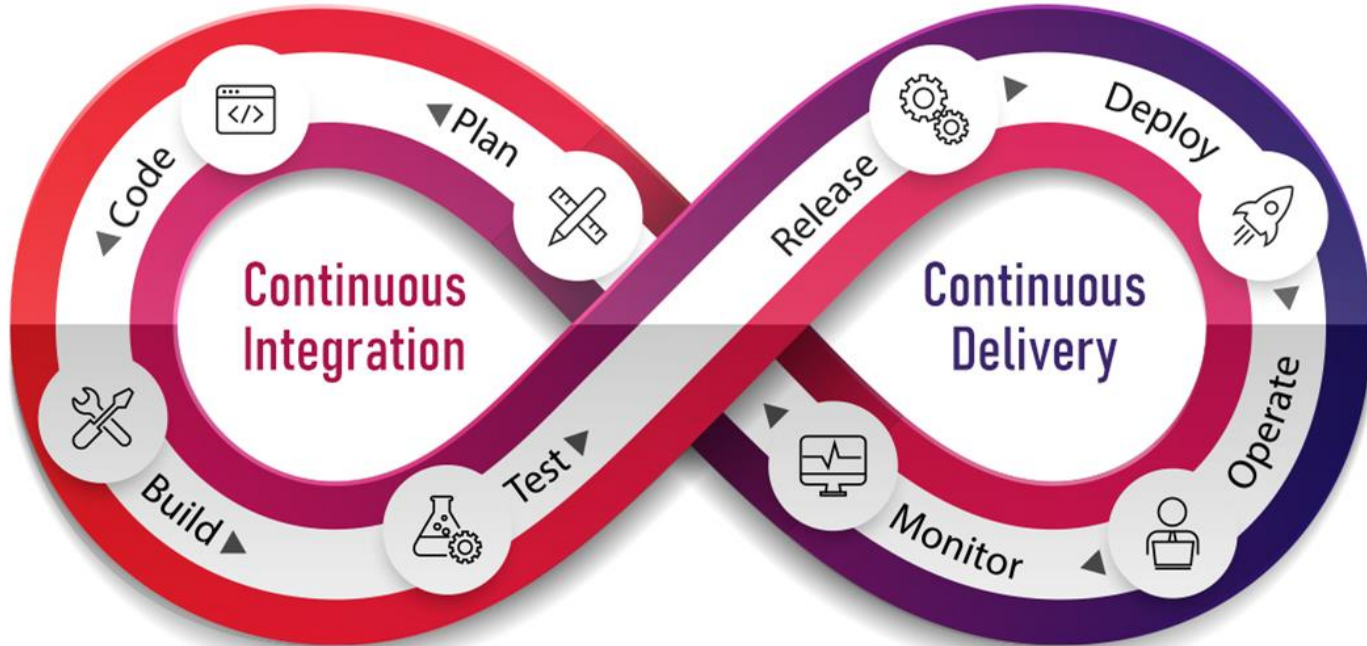
DevOps



What is DevOps Pipeline?

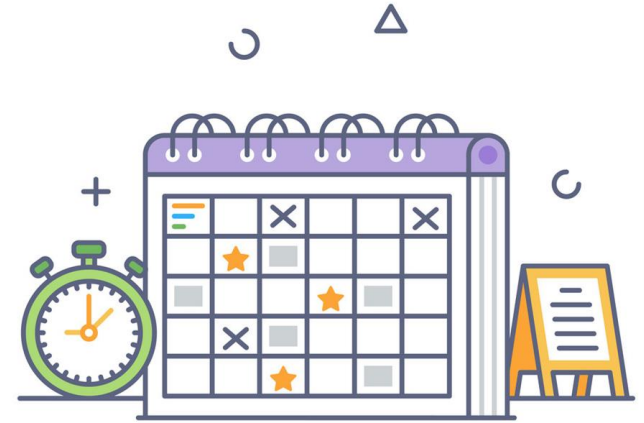
DevOps Pipeline

- According to DevOps practices, the workflow is divided into 8 phases.



1. Plan

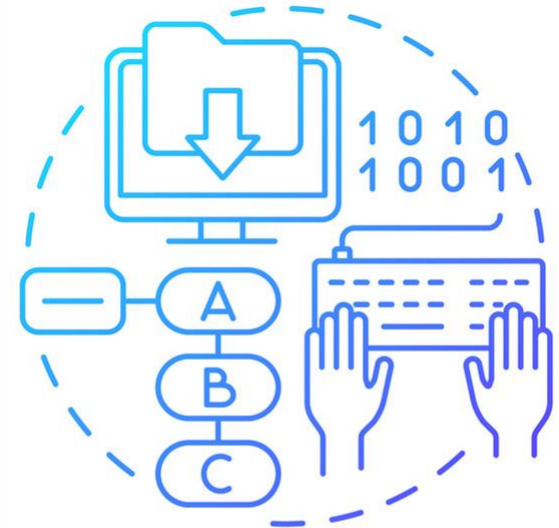
1. **Planning and prioritization of tasks:** This includes breaking down the project into smaller tasks and assigning them to specific team members.
2. **Defining objectives and goals:** This ensures that everyone is on the same page regarding what needs to be achieved and the timeline for completion.
3. **Collaborative decision-making:** This involves cross-functional team members discussing and making decisions collaboratively to ensure that the project meets quality standards.



Planning

2.Code

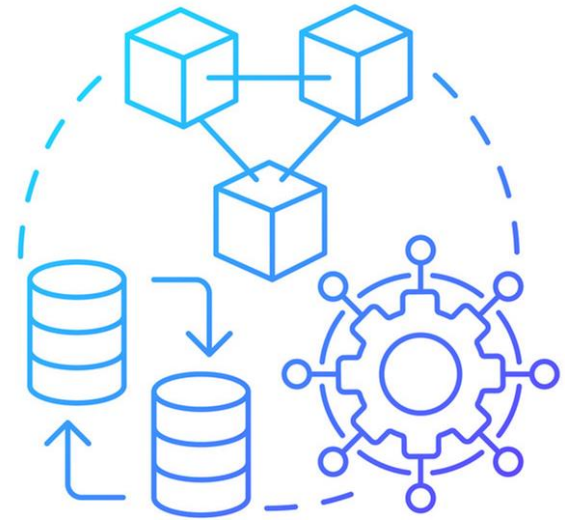
1. **Navigating the Stages of Software Development:** The Code stage involves the development, testing, and maintenance of source code for software applications.
2. **Strengthening Collaboration & Quality:** This stage incorporates tools and processes for collaboration, version control, and continuous integration to ensure the code is reliable and high quality.
3. **Solidifying Code:** Effective Code stage practices help teams produce software applications that are more stable, scalable, and secure, ultimately resulting in a better end user experience.



**LOCAL VERSION
CONTROL SYSTEM**

3.Build

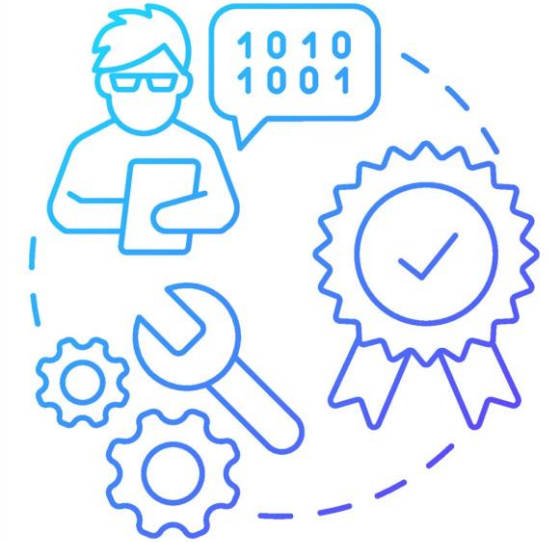
1. **From Source to Software:** Builds are the foundation of software development, as they convert source code into runnable software artifacts.
2. **Comprehensive Approach to Source Code Management:** Build stage involves compiling source code, integrating with dependencies, running automated tests, and packaging the application for deployment.
3. **The Power of Automated Builds:** Automated builds reduce errors by ensuring consistency across environments, enabling faster feedback loops, and easing the process of deployment.



PACKAGE MANAGER

4.Test

1. **Automated Testing:** It helps to ensure that all areas of the code are tested thoroughly, reducing the likelihood of bugs and other problems occurring.
2. **Continuous Testing:** This approach ensures that any issues are caught and resolved as early as possible, which can reduce the time and cost associated with fixing bugs later in the development cycle.
3. **Integration Testing:** It can help to identify any issues that arise when different components of the system are brought together, such as compatibility issues or conflicts between different software components.



**QUALITY ASSURANCE
ENGINEER**

5. Integrate (Release)

1. **Seamless Integration:** When testing is complete, new features are integrated automatically to the already existing codebase.
2. **Automated Testing and Deployment Practices:** It ensures that code changes are thoroughly tested and vetted before being released, reducing the risk of errors and downtime.
3. **Maximizing Success:** DevOps teams should prioritize communication and collaboration during the release stage to ensure that all stakeholders are aware of the changes being made and are able to provide input or feedback if necessary.



Product Release

6. Deploy

1. **Automated Deployment:** Automated deployment helps reduce errors and eliminates the need for manual deployment.
2. **Continuous Integration & Delivery:** It is a development methodology that aims to deliver software in a more incremental and iterative way.
3. **Monitoring and Feedback:** Once the code is deployed, there are additional measures in place to ensure that the application is running smoothly and delivering value to end-users.



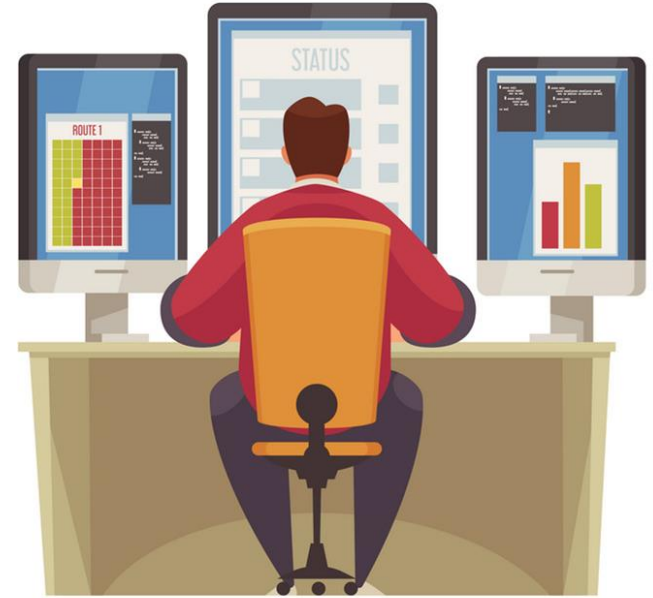
7. Operate

1. **Achieving High Availability:** The Operate Stage focuses on ensuring applications run smoothly in production and achieving high uptime to meet business goals.
2. Once the software is deployed, operations team perform activities such as configuring servers and provisioning them with the required resources.
3. Continuous Monitoring, Continuous Delivery, and Continuous Improvement are the key activities in the Operate Stage, allowing teams to identify problems early and address them proactively.

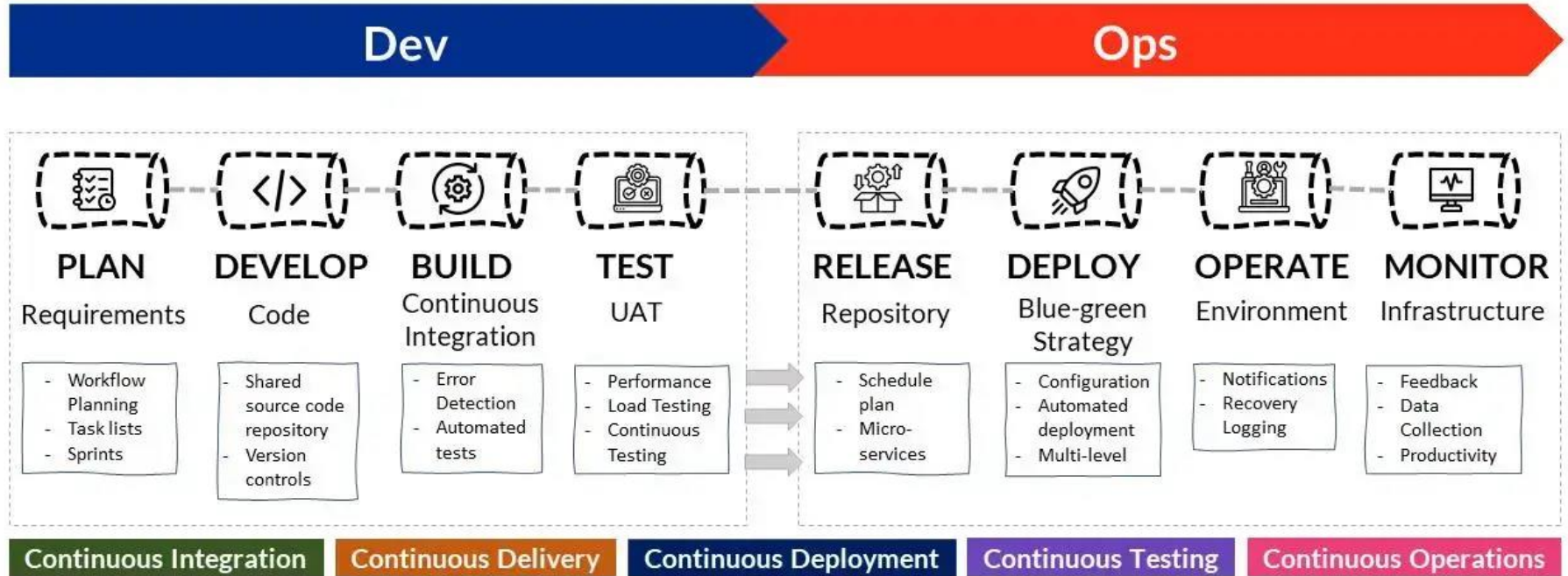


8. Monitor

1. **Continuous Monitoring:** In this stage, the team monitors various performance metrics of the deployed application continuously to ensure that the application is running smoothly. This includes monitoring metrics like server load, response time, database connectivity, and much more.
2. **Logging and Alerting:** Logs are used to capture data from different parts of the application. The data collected is analyzed to detect any unusual or abnormal activities within the application.
3. **Performance Optimization:** It enables to identify performance bottlenecks. Based on that, team can optimize the application by making code changes, upgrading hardware, or adding more resources, depending on the situation.



DevOps Pipelines



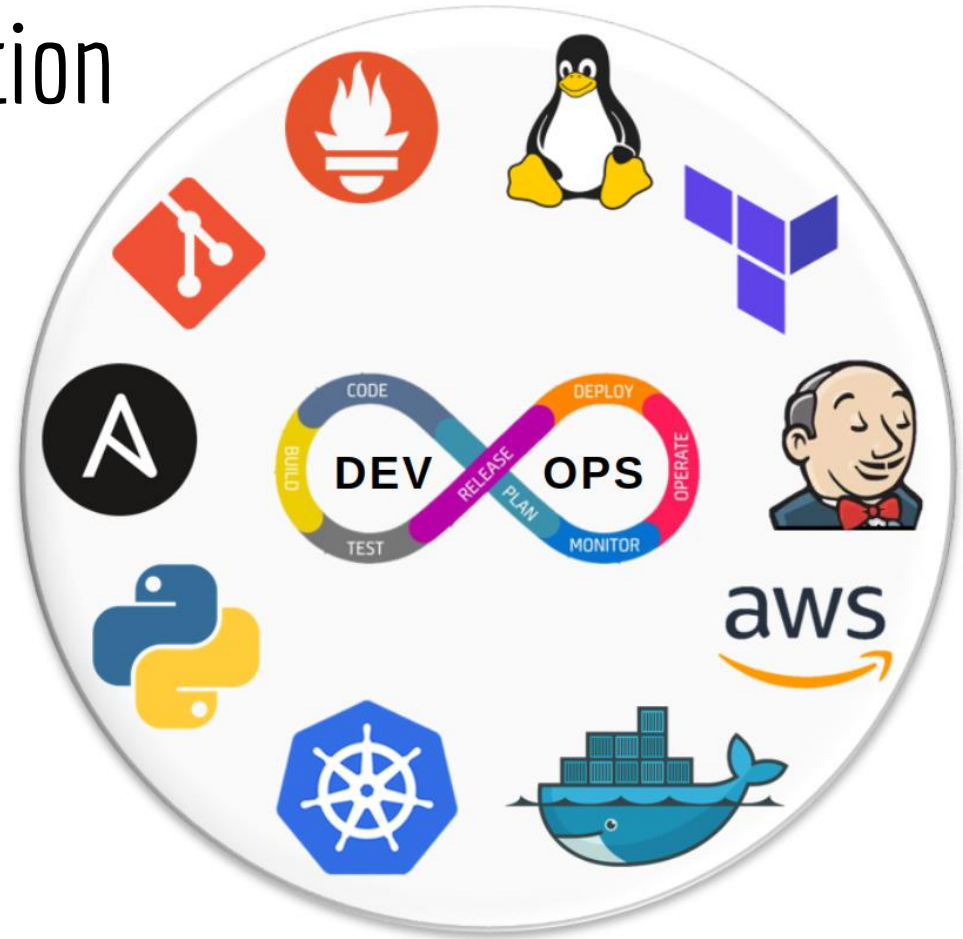


DevOps Tools Introduction



DevOps Tools Introduction

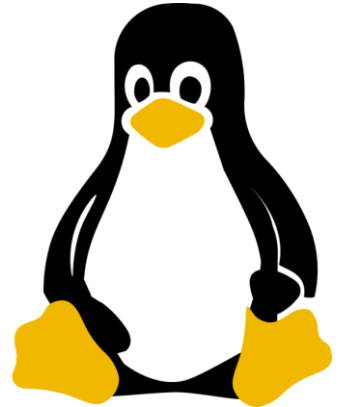
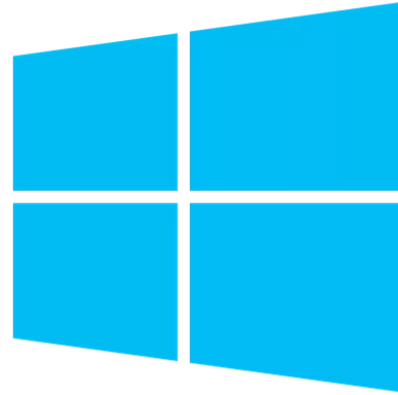
- What are DevOps Concepts?
- Which tools need to learn?
- Why need to learn these tools?
- What are DevOps tasks?
- What are DevOps responsibilities?



Operating System & Linux Basics

★ DevOps engineer should know about:

- Linux Administration
- Using CLI & Shell Commands
- Linux File Systems
- Scripting Languages



PowerShell



python



BASH
THE BOURNE-AGAIN SHELL

Networking & Security

★ DevOps engineer should know about:

- Internet Protocols (TCP/IP)
- Traffic Routings
- HTTP, HTTPS, Firewalls
- DNS Name Resolution



Concepts of Software Development

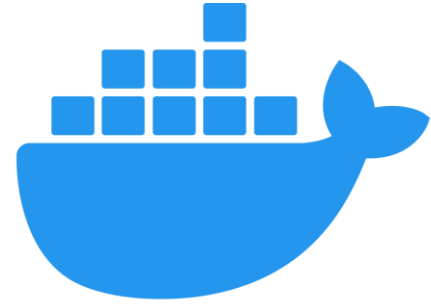
- ★ DevOps engineer should know about:
 - How developers are working together
 - Version Control Systems
 - How Git controls versioning of software
 - How to get backup from codes on Git



Containerization

★ DevOps engineer should know about:

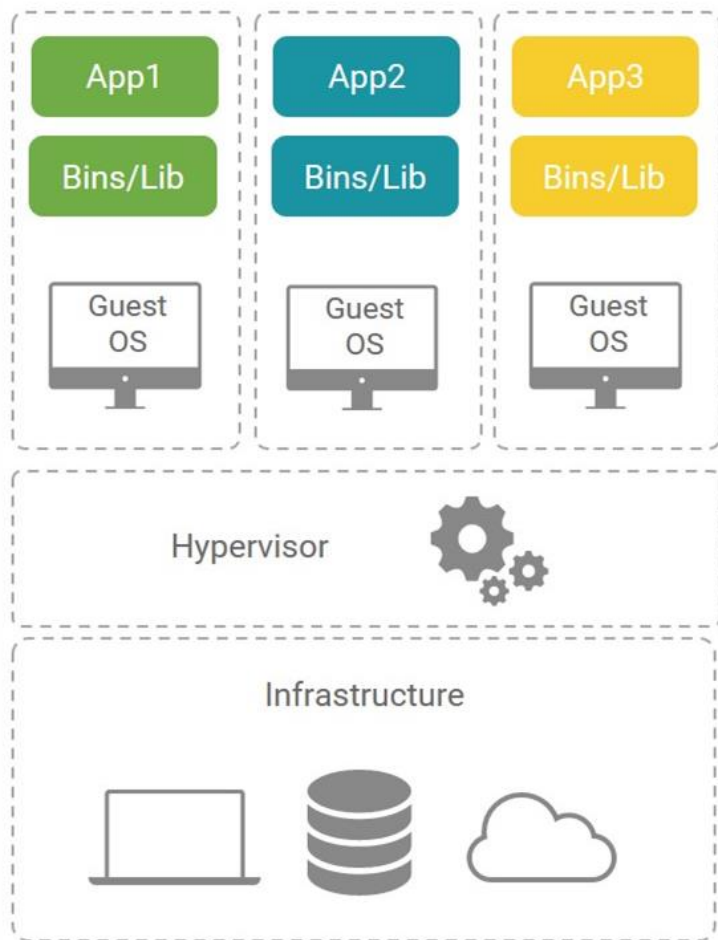
- Concepts of Virtualization
- Concepts of Containerization
- Manage Containerized Apps
- Scaling Containerized Services



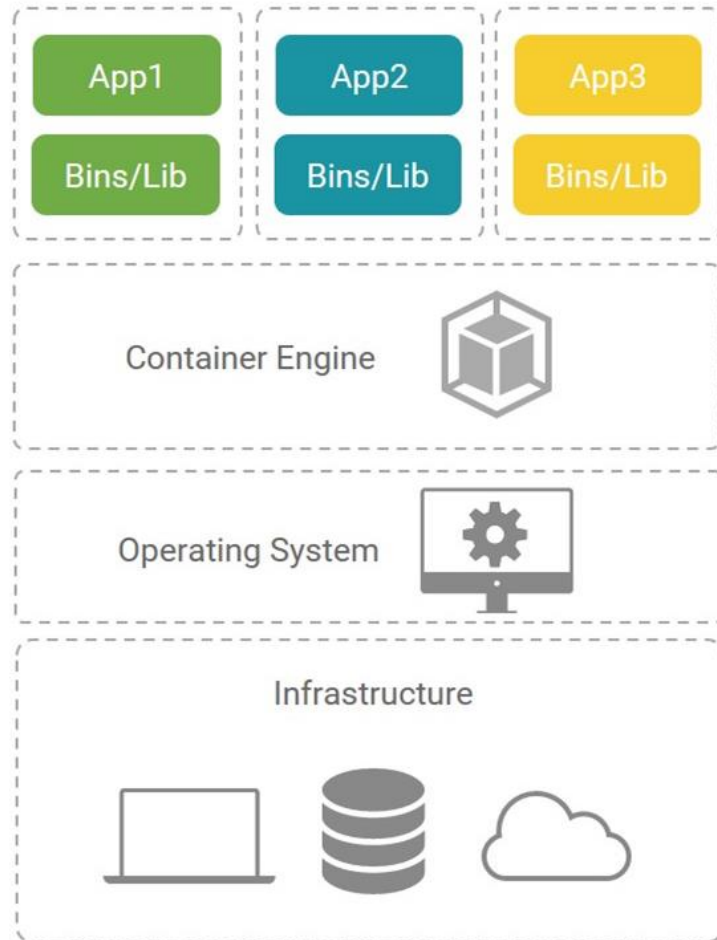
docker®



vmware®



Machine Virtualization



Containers

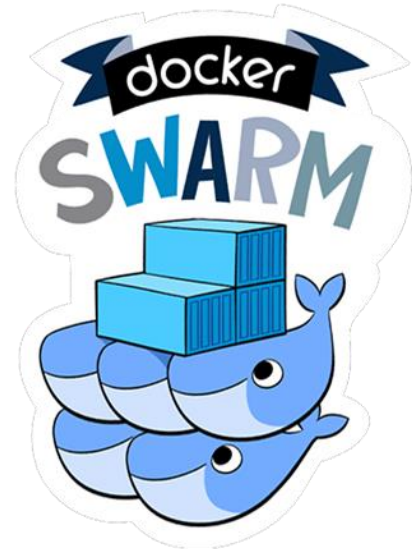
Container Orchestration

★ DevOps engineer should know about:

- Containers Need to be Managed
- How Kubernetes Works
- Manage Cluster of Containers
- How Schedule to Run Containers



kubernetes



Monitoring Tools

★ DevOps engineer should know about:

- Monitor Software
- Monitor Scripts
- Monitor Infrastructure
- Monitor Network Traffics



Grafana



PRTG
NETWORK
MONITOR

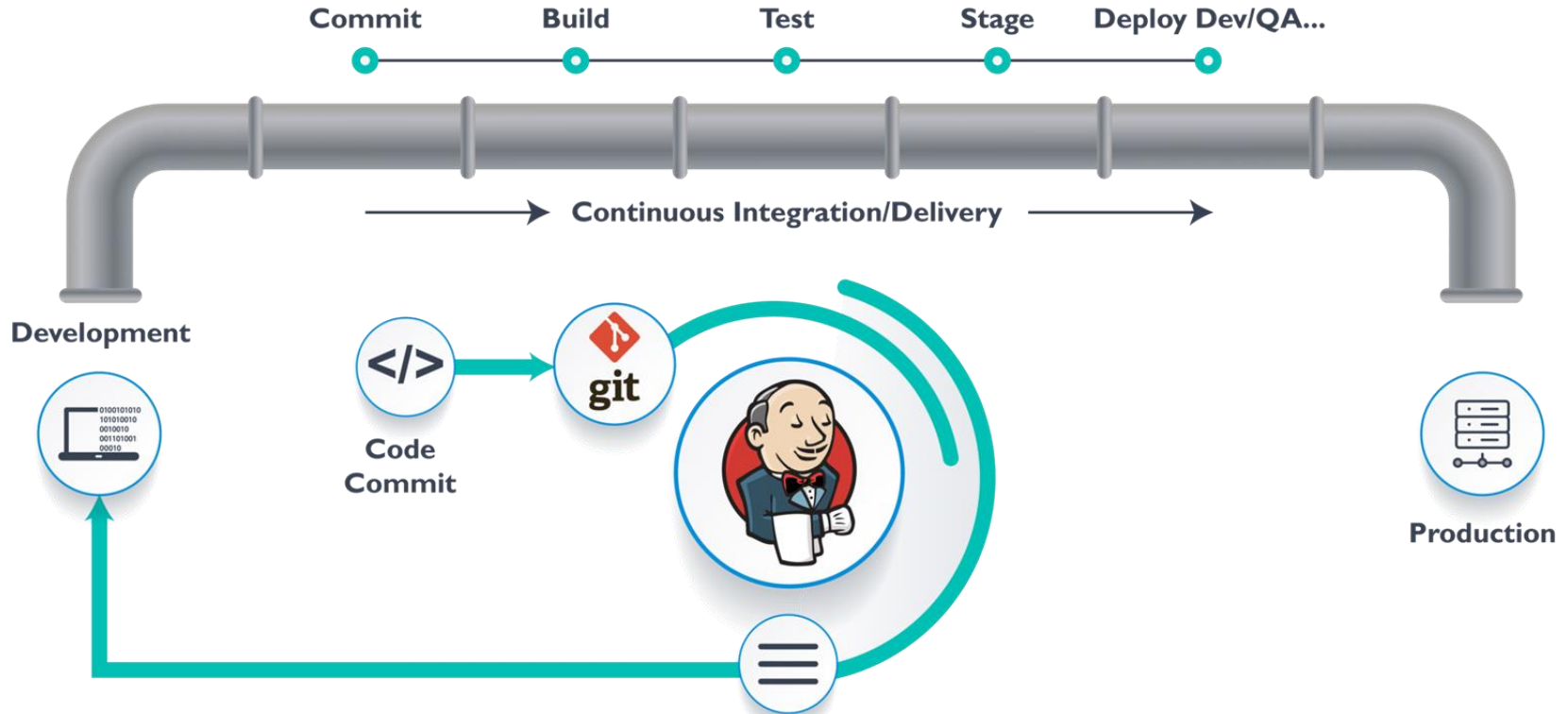
Prometheus

Infrastructure as Code (IaC)

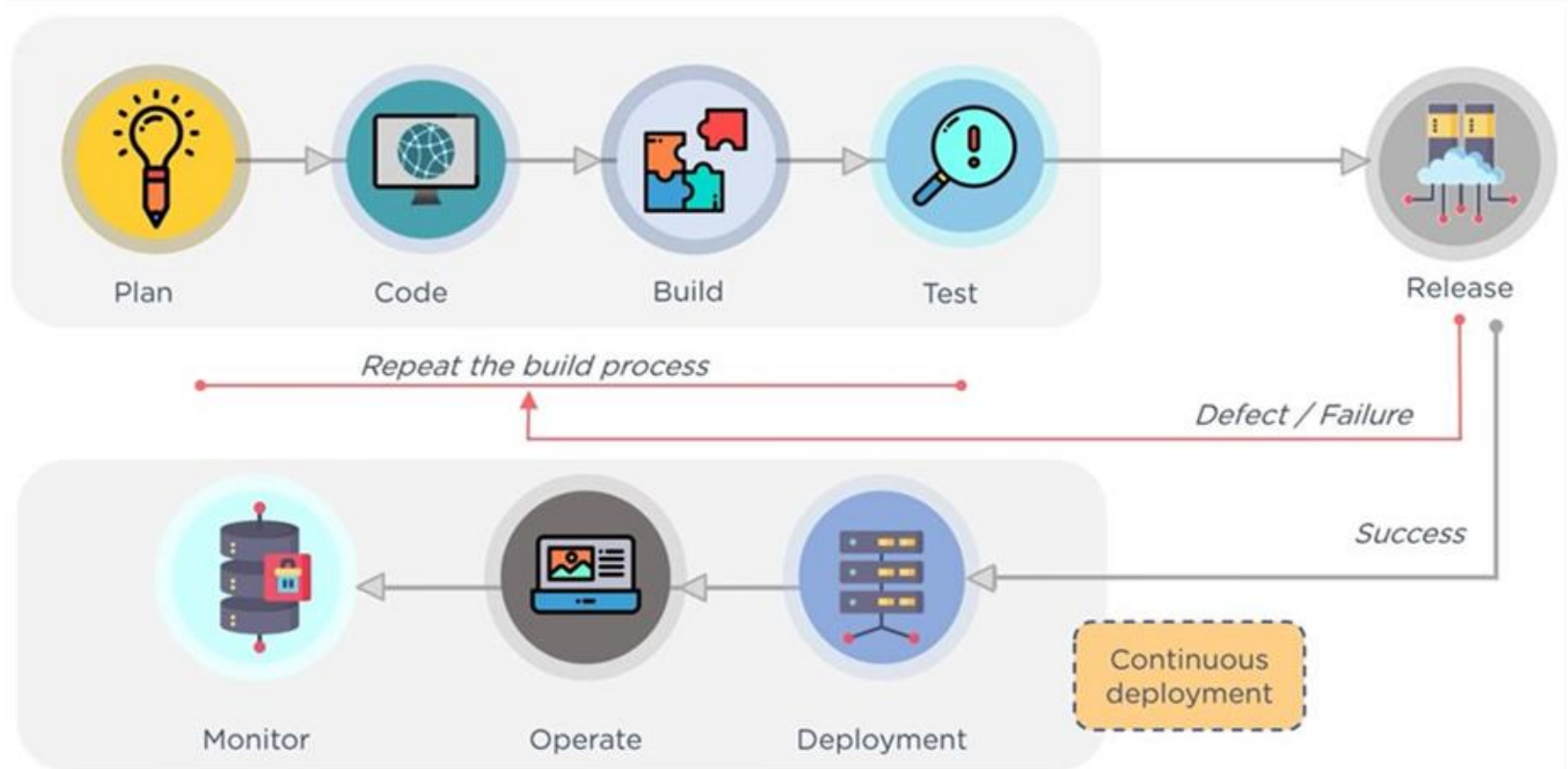
- ★ DevOps engineer should know about:
 - Configuration Management Tools
 - Use Ansible to install Dependencies
 - Use Ansible to Deploy Apps
 - Manage Live/Test/Develop Environments.



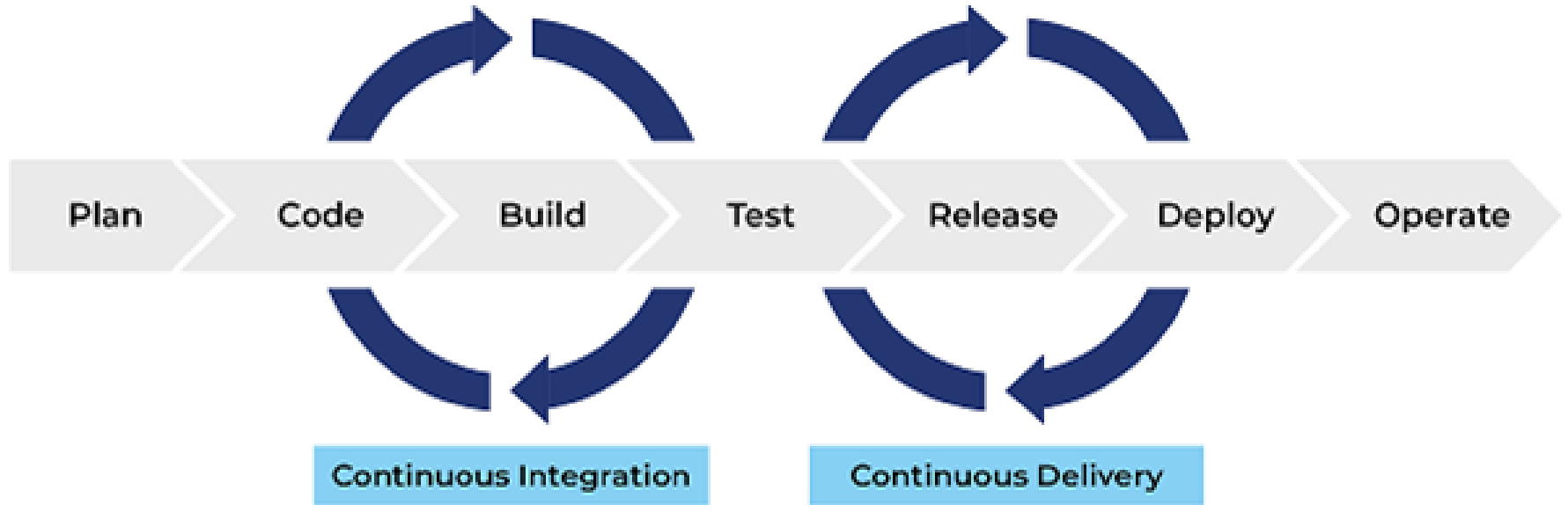
Build Automation & CI/CD

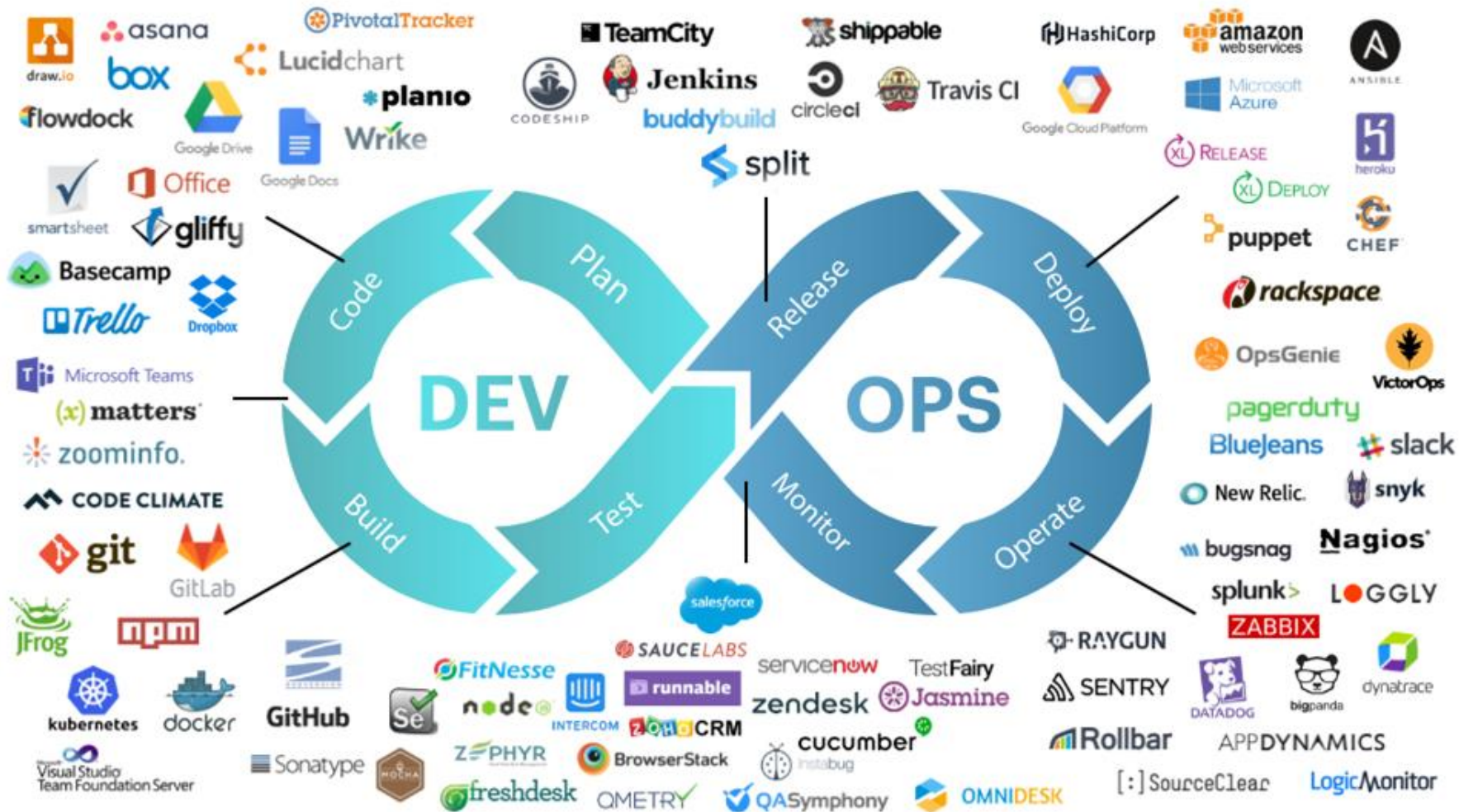


Continuous Integration / Continuous Deployment



CI/D Pipelines





Thank You!

Arash Foroughi

2023-24

SEMATEC COMPANY
